



Advanced Java Interview Mock

Coding + System Design + Architecture Thinking



Session Structure

1. Coding Round Simulation
2. Data Structures & Algorithms
3. Concurrency & JVM Questions
4. OOPS & Design Patterns
5. System Design Round
6. Behavioral & Discussion
7. Optimization & Trade-offs



Interview Round Format (Typical)

Round 1:

DSA Coding

Round 2:

Java Deep Dive

Round 3:

System Design

Round 4:

Architecture + Discussion



Coding Question 1

Design an LRU Cache

Requirements:

- `get(key)`
- `put(key, value)`
- $O(1)$ operations

Concept:

HashMap + Doubly LinkedList



LRU Cache Implementation

```
class LRUCache {  
  
    private final int capacity;  
    private java.util.Map<Integer, Node> map =  
        new java.util.HashMap<>();  
    private Node head, tail;  
  
    static class Node {  
        int key, value;  
        Node prev, next;  
  
        Node(int k, int v) {  
            key = k;  
            value = v;  
        }  
    }  
  
    public LRUCache(int capacity) {  
        this.capacity = capacity;  
    }  
}
```

```
public int get(int key) {
    if (!map.containsKey(key))
        return -1;

    Node node = map.get(key);
    remove(node);
    addToFront(node);
    return node.value;
}

public void put(int key, int value) {
    if (map.containsKey(key)) {
        Node node = map.get(key);
        node.value = value;
        remove(node);
        addToFront(node);
    } else {
        if (map.size() == capacity) {
            map.remove(tail.key);
            remove(tail);
        }
        Node node = new Node(key, value);
        addToFront(node);
        map.put(key, node);
    }
}
```



```
private void remove(Node node) {
    if (node.prev != null)
        node.prev.next = node.next;
    if (node.next != null)
        node.next.prev = node.prev;

    if (node == head) head = node.next;
    if (node == tail) tail = node.prev;
}

private void addToFront(Node node) {
    node.next = head;
    node.prev = null;

    if (head != null)
        head.prev = node;

    head = node;

    if (tail == null)
        tail = head;
}
}
```

Time:

$O(1)$



Coding Question 2

Thread-Safe Counter

Implement safe increment.

Use:

AtomicInteger or synchronized.



Concurrency Question

Explain:

- Difference between synchronized & ReentrantLock
- What is volatile?
- How does ConcurrentHashMap work?
- What is thread starvation?
- What is deadlock?



JVM Deep Dive Question

Explain:

- Heap vs Stack
- GC types
- G1 GC
- Memory leak causes
- How to debug OutOfMemoryError?



OOPS Scenario Question

Design:

Online payment system.

Discuss:

- Interfaces
- Strategy pattern
- Factory pattern
- Dependency injection



System Design Mock Question

Design a URL Shortener

Requirements:

- Generate short URLs
- Redirect efficiently
- Handle millions of requests



URL Shortener Design

Components:

- API layer
- Service layer
- Database
- Cache (Redis)
- Load balancer



Key Considerations

- Unique ID generation
- Base62 encoding
- Database indexing
- Caching frequently accessed URLs
- Horizontal scaling



Database Design

Table:

id | short_code | long_url | created_at

Index:

short_code



Scaling Strategy

- Use load balancer
- Replicate database
- Cache popular links
- Use CDN for global reach



System Design Follow-up

What if:

- 1 billion URLs?
- Need analytics?
- Need expiration?
- Need custom alias?

Discuss trade-offs.



Real Interview Follow-up

Explain:

- Time complexity of solution
- Space complexity
- Bottlenecks
- Failure handling
- Monitoring strategy



Behavioral Questions

- Describe a production issue you handled
- Explain performance optimization you did
- How do you handle high-pressure deadlines?
- How do you debug complex problems?



Optimization Mindset

In interview:

1. Start simple
2. Explain brute force
3. Improve step by step
4. Justify complexity
5. Discuss edge cases



Advanced Coding Discussion

Explain trade-offs:

HashMap vs TreeMap

ArrayList vs LinkedList

Thread pool sizing

Heap vs stack memory



Common Interview Mistakes

- Jumping to complex solution
- Ignoring edge cases
- Not explaining thought process
- Ignoring complexity
- Overengineering



Final Mock Simulation Flow

15 min:

Problem understanding

30 min:

Coding

15 min:

Optimization discussion

30 min:

System design

Remaining:

Deep Java questions



Summary

This mock prepares you for:

- Senior Java roles
- Backend engineering roles
- Architecture discussions
- System design rounds
- Production-level problem solving

Master:

- Clear thinking
- Clean coding
- Complexity explanation
- Trade-off discussion
- System-level reasoning